

PowerGym

Guia de Defesa Tecnica do Projeto

Django + PostgreSQL/Neon + Render + Cloudinary

Objetivo: Este documento foi feito para a defesa: explica o que cada ficheiro faz, como os pedidos passam por URLs, views, models e templates, como a base de dados Neon entra no sistema, como o Render faz host do site, e que decisoes tecnicas foram tomadas.

Elemento	Explicacao para a defesa
Projeto	PowerGym - Sistema de gestao de ginasio
Stack principal	Django 6, PostgreSQL/Neon, Render, Cloudinary, WhiteNoise
Data do guia	01/06/2026
Como estudar	Le por secoes, depois treina os fluxos em voz alta: login, registo, CRUD, pagamentos, deploy.

1. Explicacao curta para abrir a defesa

O PowerGym e uma aplicacao web de gestao de ginasio desenvolvida em Django. Tem landing page publica, registo e login de socios, area de cliente, dashboard de administrador, gestao de socios, treinadores, modalidades, aulas, inscricoes, planos de treino, exercicios, pagamentos e acessos.

A arquitetura segue o padrao MVT do Django: Models definem tabelas e relacoes, Views recebem pedidos e aplicam a logica, Templates mostram HTML, e URLs ligam enderecos do browser as views.

Frase segura: O pedido entra pelo browser, o urls.py escolhe a view, a view consulta models/forms, envia dados no context e o template gera a pagina final.

2. Estrutura de pastas

Elemento	Explicacao para a defesa
manage.py	Comando principal do Django: runserver, check, test, makemigrations, migrate e createsuperuser.
ginasio_project/	Configuracao global: settings.py, urls.py, wsgi.py e asgi.py.
ginasio/	Aplicacao principal: models, views, forms, urls, admin e migrations.
ginasio/templates/ginasio/	Templates HTML renderizados pelas views.
static/	CSS, logo, favicon, manifest e imagens fixas.
media/	Uploads locais em desenvolvimento; em producao as imagens vao para Cloudinary.
requirements.txt	Dependencias instaladas pelo Render no deploy.

3. Arquitetura MVT

Parte	No projeto	Responsabilidade
Model	ginasio/models.py	Define entidades e relacoes da base de dados.
View	ginasio/views.py	Recebe request, aplica regras, consulta/grava dados e devolve render ou redirect.
Template	templates/ginasio/*.html	Apresenta o HTML com variaveis vindas do context.
URLconf	ginasio/urls.py	Mapeia caminhos como /socios/ para views concretas.

Settings	ginasio_project/settings.py	Configuracao de apps, base de dados, static files, seguranca e deploy.
----------	-----------------------------	--

4. settings.py explicado

4.1 SECRET_KEY, DEBUG e dominios

```
SECRET_KEY = os.environ.get('SECRET_KEY', 'django-insecure-dev-only-change-me')
DEBUG = os.environ.get('DEBUG', 'False').lower() in ('1', 'true', 'yes', 'on')
```

A SECRET_KEY e lida das variaveis de ambiente do Render. Nao deve ficar no GitHub. DEBUG deve estar False em producao para nao mostrar erros tecnicos ao publico.

```
ALLOWED_HOSTS = 'localhost,127.0.0.1,testserver,powergym.site,www.powergym.site,.onrender.com'
```

ALLOWED_HOSTS define que dominios podem servir o site. Inclui localhost para desenvolvimento, o dominio powergym.site, www.powergym.site e o dominio tecnico do Render.

```
CSRF_TRUSTED_ORIGINS = 'https://powergym.site,https://www.powergym.site'
```

CSRF_TRUSTED_ORIGINS permite que formularios POST enviados pelo dominio HTTPS sejam aceites pelo Django.

4.2 HTTPS e cookies

```
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
SECURE_SSL_REDIRECT = os.environ.get('SECURE_SSL_REDIRECT', str(not DEBUG)).lower() in (...)
SESSION_COOKIE_SECURE = os.environ.get('SESSION_COOKIE_SECURE', str(not DEBUG)).lower() in (...)
```

O Render esta atras de um proxy. O Django precisa saber que o pedido original veio por HTTPS. As cookies de sessao e CSRF ficam seguras em producao.

4.3 INSTALLED_APPS e MIDDLEWARE

Elemento	Explicacao para a defesa
django.contrib.auth	Sistema de utilizadores, login, logout, hashes de password e permissoes.
django.contrib.sessions	Mantem o utilizador autenticado entre pedidos.
django.contrib.messages	Mostra mensagens de sucesso, erro e aviso nos templates.
cloudinary_storage/cloudinary	Uploads de imagens para Cloudinary.
django.contrib.staticfiles	Gestao de CSS, favicon e imagens estaticas.
ginasio	A aplicacao criada para o projeto.
WhiteNoiseMiddleware	Permite servir static files em producao pelo Django/Render.
CsrfViewMiddleware	Protege formularios contra ataques CSRF.
AuthenticationMiddleware	Coloca request.user disponivel nas views/templates.

4.4 Base de dados: SQLite local e Neon em producao

```
DATABASES = {
    'default': dj_database_url.config(
        default=f"sqlite:///{{BASE_DIR}}/db.sqlite3",
        conn_max_age=600
    )
}
```

O `dj_database_url` lê automaticamente a variável `DATABASE_URL`. Se existir no Render, liga ao PostgreSQL da Neon. Se não existir, usa `db.sqlite3` local. Isto permite desenvolvimento local simples e produção com PostgreSQL.

Como explicar Neon: No Render colocamos `DATABASE_URL` com a connection string da Neon. O código não contém o user/password da base de dados; lê esses dados de variáveis de ambiente.

4.5 Static files, media e Cloudinary

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

`STATICFILES_DIRS` é onde estão os ficheiros estáticos no código. `STATIC_ROOT` é para onde o `collectstatic` copia tudo no deploy.

```
STORAGES = {
    'default': {'BACKEND': 'cloudinary_storage.storage.MediaCloudinaryStorage'},
    'staticfiles': {'BACKEND': 'django.contrib.staticfiles.storage.StaticFilesStorage'}
}
```

O storage default é Cloudinary, logo os `ImageField` de `socios` e `treinadores` guardam fotos fora do Render. Isto evita perder uploads quando o servidor reinicia.

5. urls.py explicado

```
urlpatterns = [
    path('favicon.ico', RedirectView.as_view(url=settings.STATIC_URL + 'images/favicon.ico',
        permanent=True)),
    path('admin/', admin.site.urls),
    path('', include('ginasio.urls')),
]
```

`ginasio_project/urls.py` é o mapa principal. Trata `/admin/`, redireciona `/favicon.ico` para o favicon real e inclui todas as rotas da app `ginasio`.

`ginasio/urls.py` liga caminhos como `/dashboard/`, `/socios/`, `/aulas/novo/` ou `/pagamentos/<int:pk>/editar/` as respetivas views.

O que é `<int:pk>`: É o id do registo na base de dados. `/socios/5/` chama `socio_detail(request, pk=5)`.

6. models.py: entidades e relacoes

Model	Campos principais	Explicacao
-------	-------------------	------------

Socio	user, nome, email, contacto, dataNascimento, dataAdesao, foto	Ficha de cliente do ginasio; pode estar ligada a um User para login.
Treinador	user, nome, especialidade, biografia, foto	Ficha de treinador; pode estar ligada a um User para dashboard proprio.
Modalidade	nome, descricao, intensidade	Tipo de atividade: Yoga, Natacao, Pilates, etc.
Aulas	modalidade, treinador, data, hora, lotacao	Aula concreta num dia e hora, dada por um treinador.
Inscricao	socio, aula, dataRegisto	Liga socios a aulas; funciona como tabela associativa.
PlanoTreino	socio, treinador, objetivo, data_inicio	Plano prescrito por treinador a um socio.
Exercicio	planoTreino, nome, series, repeticoes, descanso_segundos	Exercicios pertencentes a um plano.
Pagamento	socio, valor, data_vencimento, data_pago, estado, frequencia, descritivo	Mensalidades e pagamentos unicos.

6.1 Relacoes

Elemento	Explicacao para a defesa
User 1-1 Socio	Permite que um socio tenha conta de login.
User 1-1 Treinador	Permite que um treinador tenha conta e dashboard.
Modalidade 1-N Aulas	Uma modalidade pode aparecer em muitas aulas.
Treinador 1-N Aulas	Um treinador pode dar muitas aulas.
Socio N-N Aulas via Inscricao	Um socio pode estar em varias aulas e uma aula pode ter varios socios.
Socio 1-N PlanoTreino	Um socio pode ter varios planos ao longo do tempo.
PlanoTreino 1-N Exercicio	Um plano tem varios exercicios.
Socio 1-N Pagamento	Um socio tem varias faturas/mensalidades.

```
Socio --< Inscricao >-- Aulas -- Modalidade
      |
      +-- Treinador
```

```
Socio --< PlanoTreino --< Exercicio
Treinador --< PlanoTreino
Socio --< Pagamento
User --1:1-- Socio / Treinador
```

As ForeignKeys usam on_delete=models.CASCADE. Se um registro pai for apagado, os filhos associados tambem podem ser removidos, mantendo a integridade da base de dados.

7. Migrations

As migrations sao o historico da estrutura da base de dados. Quando alteramos models.py, criamos migrations e depois aplicamos na base de dados.

Elemento	Explicacao para a defesa
0001_initial.py	Criou as entidades iniciais.
0002 a 0004	Evolucao do model Pagamento: datas, vencimento, frequencia e descritivo.
0005	Adicionou ligacao Socio -> User.
0006	Ajustou campos do Socio.
0007	Adicionou ligacao Treinador -> User.
0008	Ajustou Modalidade e intensidade.

```
python manage.py makemigrations
python manage.py migrate
```

8. forms.py

Os forms sao ModelForms. Um ModelForm cria automaticamente campos HTML com base no model, reduzindo repeticao.

Elemento	Explicacao para a defesa
SocioForm	Cria/edita socios; usa input date e file input para foto.
TreinadorForm	Cria/edita treinadores; aceita fotografia.
ModalidadeForm	Cria/edita modalidades.
AulasForm	Cria/edita aulas; usa date e time inputs.
InscricaoForm	Cria/edita inscricoes; dataRegisto e automatico.
PlanoTreinoForm	Cria/edita planos.
ExercicioForm	Cria/edita exercicios.
PagamentoForm	Cria/edita pagamentos; tem labels e classes Bootstrap.

Padrao de formulario: GET mostra o formulario. POST valida com form.is_valid(). Se estiver valido, form.save() grava na base de dados.

9. views.py: a logica principal

9.1 Imports importantes

```
render, redirect, get_object_or_404
User, login_required, login, messages
Socio, Treinador, Aulas, Modalidade, Inscricao, PlanoTreino, Exercicio, Pagamento
```

render devolve HTML, redirect muda de rota, get_object_or_404 procura um objeto e devolve 404 se nao existir. User e o utilizador nativo do Django.

9.2 admin_required

```
if not request.user.is_authenticated:
    return redirect('login')
if not request.user.is_superuser:
    return redirect('redirecionar_login')
return view_func(request, *args, **kwargs)
```

Este decorator protege paginas administrativas. So utilizadores autenticados e superusers podem entrar nos CRUDs.

9.3 landing_page

```
modalidades_destaque = Modalidade.objects.all()
equipa_destaque = Treinador.objects.all()
context = {'modalidades': ..., 'treinadores': ..., 'total_socios': ...}
```

A landing e publica e dinamica. Vai buscar modalidades, treinadores e contagens reais da base de dados para mostrar no site.

9.4 index dashboard

```
Pagamento.objects.filter(estado='Pendente', data_vencimento__lt=hoje).update(estado='Atrasado')
```

O dashboard admin atualiza pagamentos vencidos, calcula contagens e envia dados para graficos e cards.

9.5 Padrao CRUD

View	Exemplo	O que faz
list	socio_list	objects.all() e renderiza lista.
detail	socio_detail	get_object_or_404 pelo pk e mostra ficha.
create	socio_create	GET mostra form; POST cria.
update	socio_update	Passa instance ao form e guarda alteracoes.
delete	socio_delete	GET confirma; POST apaga.

9.6 Inscricoes sem duplicados

```
ja_existe = Inscricao.objects.filter(socio=socio_escolhido, aula=aula_escolhida).exists()
```

Antes de criar uma inscricao, o sistema confirma se o mesmo socio ja esta na mesma aula. Se estiver, mostra erro.

9.7 Cancelamento seguro de inscricoes

```
if not request.user.is_superuser:
    if not hasattr(request.user, 'socio') or inscricao.socio_id != request.user.socio.id:
        return redirect('redirecionar_login')
```

Um socio so pode cancelar as suas proprias inscricoes. O admin pode gerir todas.

9.8 Pagamentos e mensalidade automatica

```
if estado_anterior != 'Liquidado' and pagamento_guardado.estado == 'Liquidado' and
pagamento_guardado.frequencia == 'Mensal':
    Pagamento.objects.create(... estado='Pendente', frequencia='Mensal')
```

Quando uma mensalidade passa a Liquidado, o sistema cria automaticamente a proxima mensalidade pendente.

9.9 Login e redirecionamento inteligente

```
if request.user.is_superuser:
    return redirect('index')
elif hasattr(request.user, 'treinador'):
    return redirect('perfil_treinador')
else:
    return redirect('perfil_cliente')
```

Apos login, o sistema manda cada tipo de utilizador para o sitio certo: admin, treinador ou socio.

9.10 Registo de socio

```
user = User.objects.create_user(username=username, email=email, password=password)
novo_socio = Socio.objects.create(user=user, nome=nome, email=email, ...)
Pagamento.objects.create(socio=novo_socio, valor=35.00, estado='Pendente')
login(request, user)
```

O registo cria um User, cria a ficha Socio ligada ao User, cria a primeira mensalidade e inicia sessao automaticamente.

9.11 Perfil do cliente e QR Code

```
qr_data = f'SOCIO_ID:{socio.id}'
qr = qrcode.QRCode(...)
qr_base64 = base64.b64encode(buffer.getvalue()).decode()
```

O QR Code e gerado em memoria e enviado para o template em Base64. O perfil tambem mostra inscricoes, pagamentos, plano e aulas disponiveis.

9.12 Perfil do treinador

Se request.user tiver treinador associado, a view mostra apenas as aulas desse treinador com `Aulas.objects.filter(treinador=treinador)`.

9.13 Central de Acessos

Apenas superusers entram. Permite criar admin, criar treinador com conta, associar conta a treinador ou socio existente, apagar conta de login e fazer reset password.

```
user_a_alterar.set_password(nova_password)
user_a_alterar.save()
```

set_password e essencial: guarda password com hash seguro, nao em texto simples.

10. Templates

Elemento	Explicacao para a defesa
base.html	Base das paginas internas: navbar, Bootstrap, icons, CSS, favicon e estrutura comum.
landing.html	Pagina publica, com carrosseis de modalidades e treinadores vindos da base de dados.
index.html	Dashboard admin com estatisticas e graficos.
*_list.html	Listagens.
*_detail.html	Detalhes de um registo.
*_form.html	Criar/editar com forms.
*_confirm_delete.html	Confirmacao antes de apagar.
perfil_cliente.html	Area do socio.
perfil_treinador.html	Area do treinador.
gestao_acessos.html	Central de contas e passwords.

Nos templates usamos {% url 'nome_da_rota' %} para gerar links e {% static '...' %} para ficheiros estaticos. A view envia dados por context e o template usa variaveis e loops.

11. admin.py

Regista os models no /admin/ do Django. list_display escolhe colunas visiveis, search_fields permite pesquisa e list_filter adiciona filtros.

12. Deploy no Render

O GitHub guarda o codigo. O Render faz deploy automatico quando ha push. Durante o build instala dependencias, recolhe static files e aplica migracoes. Depois arranca com Unicorn.

12.1 Variaveis de ambiente

Elemento	Explicacao para a defesa
----------	--------------------------

SECRET_KEY	Chave secreta do Django.
DEBUG	False em producao.
DATABASE_URL	Connection string do Neon/PostgreSQL.
ALLOWED_HOSTS	powergym.site, www.powergym.site e .onrender.com.
CSRF_TRUSTED_ORIGINS	Dominios HTTPS aceites para POST.
CLOUD_NAME / CLOUDINARY_CLOUD_NAME	Nome da cloud Cloudinary.
API_KEY / CLOUDINARY_API_KEY	Chave da API Cloudinary.
API_SECRET / CLOUDINARY_API_SECRET	Segredo da API Cloudinary.

12.2 Comandos

Build Command:

```
pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate
```

Start Command:

```
gunicorn ginasio_project.wsgi:application
```

collectstatic prepara CSS/imagens/favicon. migrate aplica a estrutura na Neon. gunicorn corre a aplicacao Django em producao.

13. Neon

A Neon e a base PostgreSQL em producao. O Django nao tem credenciais escritas no codigo: le DATABASE_URL no Render.

```
DATABASE_URL=postgresql://user:password@host/neondb?sslmode=require...
```

Na defesa nunca mostres a password real. Explica apenas que a string contem user, password, host, nome da base e SSL.

14. Cloudinary

Cloudinary guarda imagens de socios e treinadores. Isto e melhor do que guardar uploads no disco do Render, porque o deploy pode reiniciar e nao devemos depender do filesystem local para ficheiros permanentes.

Elemento	Explicacao para a defesa
ImageField	Socio.foto e Treinador.foto.
request.FILES	Recebe ficheiros enviados por formulario.
MediaCloudinaryStorage	Envia uploads para Cloudinary.
foto.url	URL usada nos templates para mostrar a imagem.

15. Static files, favicon e WhiteNoise

Ficheiros estaticos incluem CSS, logo, favicons e manifest. WhiteNoise permite servir esses ficheiros no Render.

```
path('favicon.ico', RedirectView.as_view(url=settings.STATIC_URL + 'images/favicon.ico',
permanent=True))
```

Alguns browsers procuram /favicon.ico diretamente. Esta rota redireciona esse pedido para o favicon em /static/images/favicon.ico.

16. requirements.txt

Elemento	Explicacao para a defesa
Django	Framework web principal.
gunicorn	Servidor WSGI em producao.
dj-database-url	Converte DATABASE_URL em configuracao Django.
psycopg2-binary	Driver PostgreSQL para Neon.
whitenoise	Static files em producao.
cloudinary/django-cloudinary-storage	Uploads de imagens.
qrcode/pillow	QR Code do socio.

17. Fluxos para treinar

Registo de socio

1. Cliente abre /registo/.
2. Preenche dados e submete.
3. registo_socio verifica username/email.
4. Cria User com create_user.
5. Cria Socio ligado ao User.
6. Cria pagamento inicial pendente.
7. Faz login e redireciona para perfil_cliente.

Login

8. LoginView valida credenciais.
9. LOGIN_REDIRECT_URL chama redirecionar_login.
10. Admin vai para dashboard.
11. Treinador vai para perfil_treinador.
12. Socio vai para perfil_cliente.

Criar aula

13. Admin entra em /aulas/novo/.

14. aula_create cria AulasForm.
15. POST valida form.is_valid().
16. form.save cria a aula.
17. Redireciona para lista.

Inscrever numa aula

18. Sistema recebe socio e aula.
19. Verifica duplicado com exists().
20. Se ja existe, mostra erro/aviso.
21. Se nao existe, cria Inscricao.

Reset password

22. Admin abre Central de Acessos.
23. Seleciona utilizador e nova password.
24. View chama set_password.
25. Password fica com hash seguro.

18. Perguntas provaveis

Pergunta: Porque Django?

- Resposta: Porque ja traz ORM, autenticacao, admin, forms, migrations, templates e seguranca base.

Pergunta: O que e ORM?

- Resposta: E a camada que permite trabalhar com tabelas atraves de classes Python, como Socio.objects.filter(...).

Pergunta: Onde esta a base de dados?

- Resposta: Em producao esta na Neon/PostgreSQL, ligada pelo DATABASE_URL no Render.

Pergunta: Porque Cloudinary?

- Resposta: Para guardar uploads de imagens fora do Render, de forma permanente.

Pergunta: Como protegem admin?

- Resposta: Com admin_required e request.user.is_superuser.

Pergunta: Como distinguem roles?

- Resposta: Admin e is_superuser; treinador tem request.user.treinador; socio tem request.user.socio.

Pergunta: Password em texto simples?

- Resposta: Nao. create_user e set_password guardam hashes.

Pergunta: Para que serve migrate?

- Resposta: Aplica as migrations na base de dados.

Pergunta: Para que serve collectstatic?

- Resposta: Prepara static files para producao.

19. Melhorias finais feitas no projeto

- Landing page reformulada.

- Carrosseis de modalidades e treinadores carregados da base de dados.
- Paginas internas mais profissionais.
- Mensagem de erro no login invalido.
- Central de Acessos com reset password.
- Favicon e manifest corrigidos.
- Configuracao Render/Neon/Cloudinary por variaveis de ambiente.

20. Resumo final para memorizar

O PowerGym e uma aplicacao Django completa para gestao de ginasio. Os models representam as tabelas da base de dados e as relacoes. As views aplicam a logica, protegem acessos, usam forms e enviam dados para templates. A base de dados de producao e PostgreSQL na Neon, ligada por DATABASE_URL no Render. O Render faz deploy a partir do GitHub com collectstatic, migrate e gunicorn. O Cloudinary guarda imagens. A autenticacao usa o User do Django e separa admin, socio e treinador.